# Challenges : NumPy, Pandas, Visualisation, EDA, Cleaning, Transformation & ML

## Introduction

Ce parcours de 7 challenges mène les apprenants de la manipulation de tableaux NumPy jusqu'à un modèle de classification évalué proprement, sans fuite de données. Chaque challenge se fait en autonomie sur un notebook, avec un livrable vérifiable.

**Dataset unique du parcours : Titanic** (`sns.load_dataset('titanic')`). Tous les challenges utilisent ce même jeu de données, pour que les apprenants voient chaque étape s'enchaîner. Il contient des valeurs manquantes (`age`, `deck`), du numérique (`age`, `fare`), du nominal (`embarked`, `sex`), de l'ordinal (`pclass`) et une cible binaire (`survived`).

**Règle du parcours** : un challenge n'est validé que lorsque l'apprenant sait expliquer chaque ligne de code qu'il a écrite.

## Challenge 1 : NumPy (l'essentiel pour l'EDA)

**Objectif** : NumPy est le moteur de Pandas. On ne retient que ce qui sert à explorer des données : indexation, masques, statistiques et valeurs manquantes. Aucun code n'est fourni : les apprenants cherchent eux-mêmes la syntaxe (documentation autorisée).

| À maîtriser | Pourquoi c'est utile en EDA |
|---|---|
| `shape`, `ndim`, `dtype` | Vérifier ce qu'on manipule |
| Indexation et slicing, indices négatifs | Extraire un élément, une ligne, une colonne ou un bloc |
| Indexation par liste d'indices | Sélectionner des positions précises |
| Masques booléens | Filtrer, repérer des valeurs anormales |
| `axis=0` / `axis=1` | Calculer par colonne ou par ligne |
| `mean`, `median`, `std`, `percentile` | Résumer une distribution |
| `np.nan`, `np.isnan`, `np.nanmean` | Comprendre et compter les manquants |
| `np.where`, `np.log1p`, `np.clip` | Créer une variable, réduire l'asymétrie, plafonner |
| `np.corrcoef` | Mesurer un lien entre deux variables |

**Partie A : indexation et slicing**

Travailler sur une matrice 5×4 contenant les entiers de 0 à 19.

1. Donner sa forme, son nombre de dimensions, son type et sa taille totale.
2. Accéder à un seul élément (ligne d'indice 3, colonne d'indice 2), puis au dernier élément avec des indices négatifs.
3. Extraire la première ligne, la dernière colonne, puis la deuxième colonne entière.

4. Extraire par slicing le bloc central (lignes d'indice 1 à 3, colonnes d'indice 1 à 2).
5. Extraire une ligne sur deux, puis inverser l'ordre des lignes.
6. Sélectionner les lignes 0, 2 et 4 avec une liste d'indices. Quelle différence avec un slicing ?
7. Avec un masque booléen, sélectionner les valeurs paires supérieures à 8, puis les remplacer par 0.
8. Retrouver les positions (indices) des valeurs divisibles par 5.
9. **Vue ou copie** : modifier une valeur dans un slice et vérifier si la matrice d'origine change.

**Partie B : NumPy sur le Titanic**

1. Charger le Titanic et extraire `age` et `fare` sous forme de tableaux NumPy.
2. Compter les valeurs manquantes de `age`. Comparer la moyenne simple et la moyenne qui ignore les NaN : que se passe-t-il ?
3. Calculer moyenne, médiane, écart-type, minimum, maximum et percentiles 25 et 75 de `age` et de `fare`. Comparer moyenne et médiane de `fare` : que révèle l'écart ?
4. Calculer l'IQR de `fare`, la borne supérieure `Q3 + 1.5 × IQR`, puis compter les outliers avec un masque.
5. Appliquer `np.log1p` à `fare` et comparer la distribution avant et après (moyenne, médiane).
6. Créer un tableau « enfant / adulte » (seuil de 12 ans) et compter les enfants.
7. Calculer la corrélation entre `age` et `fare` (attention aux NaN) et l'interpréter.

**Questions de compréhension**

- Quelle différence entre `axis=0` et `axis=1` ?
- Qu'est-ce qu'une vue, qu'est-ce qu'une copie, et pourquoi cela compte-t-il quand on modifie un tableau ?
- Pourquoi la médiane est-elle plus fiable que la moyenne pour `fare` ?

# Challenge 2 : Pandas (l'essentiel pour l'EDA)

**Objectif** : maîtriser les Series, les conditions logiques et les outils DataFrame qui suffisent pour mener une EDA complète. Dataset : Titanic.

**Partie A : les Series**

Une colonne de DataFrame est une Series : un tableau à une dimension avec un index et un nom.

1. Extraire la colonne `age` du Titanic sous forme de Series. Quelle différence avec l'extraction de `age` sous forme de DataFrame (double crochets) ?
2. Accéder à des éléments avec `loc` (étiquette) et `iloc` (position). Donner un cas où les deux donnent des résultats différents.
3. Faire des opérations vectorisées : `fare` en centaines, `age` arrondi, somme de `sibsp` et `parch`.
4. Explorer une Series : `unique`, `nunique`, `value_counts` (avec et sans `normalize`), `min`, `max`, `idxmax`, `sort_values`, `nlargest`.
5. Détecter et traiter les manquants d'une Series : `isna`, `notna`, `fillna`, `dropna`.
6. Transformer : `astype`, `map` avec un dictionnaire (par exemple `male` en `0` et `female` en `1`), `apply` avec une petite fonction.

**Partie B : conditions logiques et filtres**

Une comparaison sur une Series produit une Series de booléens (un masque). C'est la base de tout filtrage.

1. Écrire une condition simple (`age` supérieur à 30) et observer le résultat. Combien de `True` ? Quelle proportion ?
2. Combiner des conditions avec `&` (et), `|` (ou), `~` (non). Pourquoi faut-il des parenthèses ? Pourquoi `and` / `or` ne fonctionnent-ils pas ?
3. Utiliser `isin`, `between`, `isna`, `notna` pour écrire : passagers embarqués à C ou Q, tarif entre 10 et 50, âge manquant.
4. Filtrer le DataFrame avec le masque, puis avec `loc` (masque + choix de colonnes), puis avec `query`. Comparer les trois écritures.
5. Compter et mesurer avec un masque : somme = nombre de lignes, moyenne = proportion. Calculer ainsi le taux de survie global.
6. Créer une colonne conditionnelle avec `np.where` (par exemple `enfant` si `age` < 12), puis une version à plusieurs cas avec `np.select`.
7. Répondre avec des conditions logiques : combien de femmes en 1re classe ? Quel est leur taux de survie ? Combien d'hommes de plus de 60 ans ? Combien de passagers voyageant seuls en 3e classe ?

**Partie C : les outils DataFrame pour l'EDA**

| Étape d'EDA | Outils Pandas |
|---|---|
| Inspecter | `head`, `shape`, `info`, `describe(include='all')`, `dtypes` |
| Qualité | `isna().mean()`, `duplicated().sum()`, `nunique()` |
| Distribution | `value_counts(normalize=True)`, `skew()`, `quantile` |
| Créer des variables | `pd.cut`, `pd.qcut`, `assign`, `str.extract` |
| Comparer des groupes | `groupby().agg()`, `pivot_table`, `crosstab` |
| Corrélations | `df.corr(numeric_only=True)` |

1. Inspecter le dataset : nombre de lignes et de colonnes, types, % de manquants par colonne.
2. Séparer les colonnes en numériques, nominales et ordinale.
3. **groupby** : taux de survie par sexe, par classe, puis par sexe et classe, avec `agg` (`mean` et `count`).
4. Créer une tranche d'âge avec `pd.cut` et regarder la survie par tranche.
5. Calculer la matrice de corrélation et identifier les variables numériques les plus liées à la cible.

**Questions de compréhension**

- Quelle différence entre une Series et un DataFrame ?
- Pourquoi `df[df.age > 30]['sex'] = 'x'` déclenche un `SettingWithCopyWarning`, et que faire à la place ?
- Pourquoi la moyenne d'un masque booléen donne-t-elle une proportion ?

## Challenge 3 : Visualisation (Matplotlib et Seaborn)

**Objectif** : choisir le bon graphique pour la bonne question, le rendre lisible, et surtout **savoir l'interpréter**. Dataset : Titanic.

**Règle d'or** : un graphique sans interprétation ne vaut rien. Pour chaque visuel, l'apprenant écrit 3 phrases :

1. **Ce que je vois** : décrire la forme, les valeurs clés, les écarts.

2. **Ce que cela signifie** : la conclusion pour la question posée.
3. **Limite ou piste** : ce que le graphique ne prouve pas, ou ce qu'il faut vérifier ensuite.

**Partie A : Matplotlib**

Comprendre l'objet `Figure` / `Axes`, puis produire chaque graphique avec titre, labels d'axes, légende si besoin .

| Tâche |
|---|
| Histogramme de `age` (essayer 10, 20 et 40 classes) |
| Barplot du nombre de passagers par classe |
| Nuage de points `age` contre `fare` |
| Grille 2×2 regroupant 4 graphiques |

**Partie B : Seaborn**

Pour chaque question : choisir et produire le graphique, puis écrire l'interprétation en 3 phrases. Comparer ensuite avec la piste.

| | Question | Graphique conseillé |
|---|---|---|
| 1 | Comment l'âge est-il distribué ? | `histplot` avec `kde` |
| 2 | Comment le tarif est-il distribué ? | `histplot` |
| 3 | Y a-t-il des outliers dans le tarif, selon la classe ? | `boxplot` de `fare` par `pclass` |
| 4 | La cible est-elle équilibrée ? | `countplot` de `survived` |
| 5 | Le sexe influence-t-il la survie ? | `barplot` de `survived` par `sex` |
| 6 | La classe influence-t-elle la survie ? | `barplot` de `survived` par `pclass` |
| 7 | L'effet de la classe est-il le même selon le sexe ? | `catplot` ou `pointplot` avec `hue` |
| 8 | L'âge distingue-t-il les survivants ? | `violinplot` ou `boxplot` de `age` par `survived` |
| 9 | Y a-t-il un lien entre âge et tarif, selon la classe ? | `scatterplot` avec `hue='pclass'` |
| 10 | Quelles variables sont liées entre elles et à la cible ? | `heatmap` de la matrice de corrélation (`annot=True`, `center=0`) |
| 11 | Où sont les valeurs manquantes ? | `heatmap` de `isna()` |
| 12 | Quelles relations croisées se détachent d'un coup d'œil ? | `pairplot` avec `hue='survived'` |

**Partie C : le défi du jugement**

1. Reprendre trois graphiques de la partie B et les refaire en changeant volontairement un choix (échelle de l'axe, nombre de classes, type de graphique). Qu'est-ce qui change dans l'interprétation ?
2. Trouver un graphique qui peut tromper (axe tronqué, barres sans effectifs, moyenne cachant une distribution) et expliquer pourquoi.
3. Écrire la phrase de conclusion de l'ensemble : quelles sont les 3 variables les plus importantes pour prédire la survie, et pourquoi ?

**Critères de qualité** : titre explicite, axes nommés avec unités, légende utile, pas de couleurs inutiles, taille de police lisible, un seul message par graphique.

## Challenge 4 : EDA (analyse exploratoire du Titanic)

**Objectif** : comprendre le Titanic avant de modéliser, en répondant à des questions précises, dans l'ordre, avec des chiffres. Chaque étape produit des constats chiffrés et des décisions pour la suite. Aucun code n'est fourni.

**Consigne générale** : pour chaque question, l'apprenant écrit la réponse chiffrée, le graphique ou le tableau utilisé, puis une phrase de conclusion (« donc... »).

| Étape | Questions précises à traiter |
|---|---|
| 1. Cadrage | Quelle est la cible et de quel type ? Combien de lignes et de colonnes ? Classer chaque colonne en numérique, nominale, ordinale ou booléenne. Quelles colonnes disent la même chose ? |
| 2. Qualité | Quel % de manquants par colonne ? Quelles colonnes sont inutilisables ? Combien de lignes dupliquées, et faut-il les supprimer ? Y a-t-il des valeurs impossibles ? |
| 3. Univariée numérique | Pour `age` et `fare` : moyenne, médiane, minimum, maximum, asymétrie ? Combien d'outliers de tarif selon l'IQR ? Quelle transformation envisager ? |
| 4. Univariée catégorielle | Répartition de `embarked`, `pclass`, `sex`, `alone` ? Quelle modalité est rare ? |
| 5. Cible | La cible est-elle équilibrée ? |
| 6. Bivariée avec la cible | Taux de survie par sexe, par classe, par `who` (homme, femme, enfant) ? Que valent les corrélations de chaque variable numérique avec `survived` ? |
| 7. Redondance | Quelles variables sont fortement corrélées entre elles ? Lesquelles dériveraient d'autres ? |
| 8. Multivariée | Taux de survie selon sexe et classe ensemble ? Y a-t-il une interaction ? |
| 9. Manquants et cible | Les âges manquants sont-ils liés à la classe ou à la survie ? |

**Étape 10 : la synthèse décisionnelle**

À partir des constats ci-dessus, l'apprenant remplit un tableau de décisions :

| Décision | À justifier |
|---|---|
| Colonnes à retirer | `alive` (fuite : c'est la cible), `class` et `embark_town` (doublons de `pclass` et `embarked`), `deck` (77 % de manquants) |
| Colonnes à retraiter | `fare` (asymétrie), `age` (imputation, éventuellement par groupe), `who` et `adult_male` (dérivées de `sex` et `age` : garder ou non ?) |
| Types de transformation | Numérique : imputation + mise à l'échelle ; nominal : one-hot ; ordinal : encodage ordonné pour `pclass` |
| Métrique à privilégier | Pas l'accuracy seule : F1, recall et precision de la classe positive |

**Questions de réflexion**

- Quelles variables semblent les plus liées à la cible, et lesquelles sont redondantes ?
- Quelles variables ne devraient pas être utilisées pour prédire, et pourquoi ?
- Corrélation n'est pas causalité : donner un exemple trouvé dans le Titanic (par exemple le lien entre `fare` et survie passe par la classe).

## Challenge 5 : Cleaning

**Objectif** : transformer des données sales en données fiables, en justifiant chaque décision. Dataset : Titanic, que le formateur peut abîmer volontairement (espaces, casse incohérente, dates en texte, valeurs `'?'`, doublons).

**Consignes**

1. **Manquants** : mesurer avec `df.isna().mean()`. Pour chaque colonne, choisir et justifier une stratégie : suppression de la ligne, suppression de la colonne, imputation par médiane, par mode, par groupe (`groupby().transform('median')`) ou ajout d'un indicateur `age_manquant`.
2. **Doublons** : détecter (`duplicated`), inspecter avant de supprimer, puis `drop_duplicates` avec le bon `subset`.
3. **Types** : convertir avec `astype`, `pd.to_numeric(errors='coerce')`, `pd.to_datetime`, et passer les colonnes à faible cardinalité en `category`.
4. **Texte** : `str.strip`, `str.lower`, `str.replace`, harmoniser les modalités (`'M'`, `'male'`, `'Male '` deviennent une seule valeur).
5. **Outliers** : détecter par la règle IQR (`Q1 - 1.5 × IQR`, `Q3 + 1.5 × IQR`) et par z-score ; décider : garder, plafonner (winsorisation) ou supprimer, en fonction du contexte métier.
6. **Incohérences** : repérer les valeurs impossibles (âge négatif, date de sortie avant la date d'entrée) avec des contrôles logiques.
7. Renommer les colonnes en `snake_case` et écrire une fonction `clean(df)` reproductible qui applique toutes les étapes.

**Questions de compréhension**

- Pourquoi la moyenne est-elle un mauvais choix d'imputation sur une variable très asymétrique ?
- Quand un outlier est-il une erreur et quand est-il une information ?

- Pourquoi faut-il calculer les valeurs d'imputation sur le jeu d'entraînement uniquement ? (introduction au challenge 7)

## Challenge 6 : Transformation (fit, transform, fit_transform, inverse_transform)

**Objectif** : maîtriser le cycle des transformateurs scikit-learn et choisir la bonne transformation selon le type de variable.

**Partie A : comprendre les 4 méthodes**

| Méthode | Ce qu'elle fait | Sur quel jeu ? |
|---|---|---|
| `fit(X_train)` | Apprend les paramètres (moyenne et écart-type, min et max, catégories, médiane) | Entraînement uniquement |
| `transform(X)` | Applique les paramètres déjà appris | Entraînement, validation et test |
| `fit_transform(X_train)` | `fit` puis `transform` en un appel | Entraînement uniquement |
| `inverse_transform(X)` | Revient à l'échelle ou aux catégories d'origine | Valeurs transformées ou prédictions |

**À retenir** : les attributs appris par un transformateur se terminent par `_` (par exemple `mean_`, `scale_`, `categories_`). Après un `fit`, les afficher permet de voir exactement ce qui a été appris sur le jeu d'entraînement. Les apprenants écrivent eux-mêmes le code de la partie A.

**Exercices**

1. Faire un `fit_transform` sur le test puis comparer les moyennes avec la bonne méthode : constater que `mean_` change, et expliquer pourquoi c'est une erreur.
2. Vérifier que `inverse_transform(transform(X))` redonne `X` (`np.allclose`).
3. Lister les attributs appris (`mean_`, `scale_`, `categories_`, `statistics_`) pour chaque transformateur.

**Partie B : choisir selon le type de variable**

| Type | Exemple (Titanic) | Transformateur | Remarque |
|---|---|---|---|
| Numérique continue | `age`, `fare` | `StandardScaler`, `MinMaxScaler`, `RobustScaler` | `RobustScaler` si outliers ; `PowerTransformer` ou `log1p` si très asymétrique |
| Nominale (sans ordre) | `embarked`, `sex` | `OneHotEncoder(handle_unknown='ignore')` | Pas de `LabelEncoder` sur les features : il impose un ordre qui n'existe pas |

| Type | Exemple (Titanic) | Transformateur | Remarque |
|---|---|---|---|
| Ordinale (avec ordre) | `pclass` : 3 < 2 < 1, niveau d'études | `OrdinalEncoder(categories=[[...]])` | Toujours donner l'ordre explicitement |
| Cible | `survived` | `LabelEncoder` | Réservé à `y` ; c'est ici que `inverse_transform` sert à relire les prédictions |
| Manquants | `age`, `deck` | `SimpleImputer(strategy='median')`, `'most_frequent'` | Se calcule sur le train uniquement |

**Partie C : mise en pratique**

1. Séparer d'abord les colonnes en trois listes : `num_cols`, `nom_cols`, `ord_cols`.
2. Appliquer chaque transformateur séparément et observer la shape avant et après (un `OneHotEncoder` multiplie les colonnes).
3. Pour `OrdinalEncoder`, définir `categories=[['Low', 'Medium', 'High']]` sur un exemple, et vérifier que `Low < Medium < High` donne bien 0, 1, 2.
4. Tester un `OneHotEncoder` sur une modalité absente du train : provoquer l'erreur, puis la corriger avec `handle_unknown='ignore'`.
5. Utiliser `get_feature_names_out()` pour retrouver les noms des colonnes encodées.
6. Combiner le tout dans un `ColumnTransformer` (préparation du challenge 7).

**Questions de compréhension**

- Pourquoi un arbre de décision n'a-t-il pas besoin de standardisation alors qu'une régression logistique ou un KNN en ont besoin ?
- Que se passe-t-il si on standardise avant de séparer train et test ?
- Quand utiliser `inverse_transform` en pratique ?

## Challenge 7 : ML classification (de la donnée brute au test final)

**Objectif** : construire un modèle de classification complet, reproductible, sans fuite de données, et interpréter ses résultats. Dataset : Titanic (cible `survived`).

**Le flux de travail à respecter**

1. Définir `X` et `y`, retirer les colonnes interdites (identifiants, colonnes qui révèlent la cible).
2. **Séparer d'abord** : `train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)`. Le jeu de test n'est plus touché jusqu'à l'étape 8.
3. Construire le `ColumnTransformer` (imputation + standardisation du numérique, imputation + `OneHotEncoder` du nominal, `OrdinalEncoder` de l'ordinal).
4. Construire le `Pipeline` : préparation puis modèle.
5. **Cross-validation** du pipeline sur le train (`StratifiedKFold`, `cross_val_score` ou `cross_validate`).
6. Comparer 2 ou 3 modèles (régression logistique, arbre, random forest).

7. **GridSearchCV** sur le meilleur candidat, puis analyse des résultats.
8. Évaluer **une seule fois** sur le test, puis interpréter.

**Partie A : le data leakage**

Le data leakage (fuite de données) arrive quand une information du jeu de test, ou de la cible, influence l'entraînement. Le modèle paraît excellent mais échoue en production.

| Faute fréquente | Pourquoi c'est une fuite | Correction |
|---|---|---|
| `scaler.fit_transform(X)` avant le split | La moyenne et l'écart-type incluent le test | Split d'abord, `fit` sur le train |
| Imputer la médiane sur tout le jeu | La médiane utilise des valeurs du test | `SimpleImputer` dans le pipeline |
| Sélection de variables sur tout le jeu | Les variables sont choisies avec les labels du test | La sélection entre dans le pipeline |
| Standardiser avant la cross-validation | Chaque pli voit les stats des autres plis | Pipeline dans `cross_val_score` |
| Variable qui contient la cible (ex. `alive` pour `survived`) | Elle est déduite de la réponse | La retirer après l'EDA |
| Régler les hyperparamètres sur le test | Le test devient un jeu de validation | GridSearch sur le train, test utilisé une fois |

**Exercice de démonstration** : refaire l'expérience deux fois, une fois avec un `StandardScaler` ajusté sur tout le jeu avant le split, une fois avec le pipeline correct. Comparer les scores et discuter de l'écart (faible sur Titanic, mais le principe reste).

**Partie B : le pipeline**

**Architecture à construire**

Colonnes proposées pour le Titanic : numériques `age`, `fare`, `sibsp`, `parch` ; nominales `sex`, `embarked` ; ordinale `pclass`. Colonnes exclues : `alive`, `class`, `embark_town`, `deck`.

- Pipeline numérique : imputation par la médiane, puis standardisation.
- Pipeline nominal : imputation par la modalité la plus fréquente, puis one-hot encoding qui ignore les modalités inconnues.
- Pipeline ordinal : imputation par la modalité la plus fréquente, puis encodage ordinal avec l'ordre explicite (3e, 2e, 1re classe).
- Un transformateur de colonnes qui applique chaque pipeline à ses colonnes.
- Un pipeline final : préparation suivie du modèle de classification (commencer par la régression logistique).

Points à faire expliquer : pourquoi tout est dans un seul objet, ce que fait `model.fit(X_train, y_train)` à l'intérieur, et pourquoi `model.predict(X_test)` ne recalcule aucun paramètre.

**Partie C : cross-validation et GridSearch**

### À réaliser

1. Définir une validation croisée stratifiée à 5 plis, avec mélange et graine fixée.
2. Calculer le score F1 en validation croisée du **pipeline entier** (pas des données déjà transformées), puis afficher la moyenne et l'écart-type.
3. Lancer une recherche par grille sur le pipeline avec une grille sur le paramètre de régularisation `C` et sur `class_weight`. Les noms de paramètres suivent le format `étape__paramètre`.
4. Afficher les meilleurs paramètres et le meilleur score de validation.

### Partie D : les métriques de classification

Pour une classe positive (ex. « a survécu », « tumeur maligne »), la matrice de confusion compte quatre cas : VP (vrais positifs), FP (faux positifs), FN (faux négatifs), VN (vrais négatifs).

| Métrique | Formule | Question à laquelle elle répond |
|---|---|---|
| Accuracy | (VP + VN) / total | Quelle part des prédictions est correcte ? |
| Precision | VP / (VP + FP) | Quand le modèle annonce « positif », a-t-il raison ? |
| Recall (sensibilité) | VP / (VP + FN) | Parmi les vrais positifs, combien en a-t-on trouvés ? |
| F1-score | 2 × P × R / (P + R) | Compromis entre précision et rappel |
| Spécificité | VN / (VN + FP) | Parmi les vrais négatifs, combien sont bien écartés ? |

### À réaliser

1. Prédire sur le jeu de test avec le meilleur modèle.
2. Afficher le rapport de classification (precision, recall, F1, support).
3. Afficher la matrice de confusion sous forme de graphique.

### Exercices d'interprétation

1. Sur un jeu à 95 % de négatifs, un modèle qui répond toujours « négatif » a 95 % d'accuracy : calculer son recall et son F1 sur la classe positive. Conclure sur le piège de l'accuracy.
2. Remplir à la main une matrice de confusion avec VP = 40, FP = 10, FN = 20, VN = 130, puis calculer accuracy, precision, recall et F1.
3. Cas métier : dépistage d'une maladie grave (privilégier le recall) et filtre anti-spam (privilégier la precision). Justifier le choix de la métrique et du seuil.
4. Faire varier le seuil de décision (`predict_proba` > 0,3 / 0,5 / 0,7) et observer le compromis precision/recall.
5. Lire les `macro avg`, `weighted avg` et `support` du `classification_report`.
6. Comparer l'écart entre le score de cross-validation et le score de test : un écart important signale un surapprentissage ou une fuite.